Nama: Joshua Taftfarel Petra Chrisensa
NPM: 232310002
Kelas: TI-23-KA

.env

```env
API_PORT=3002
NODE_ENV=production

// MySQL Configuration
EXPRESS_MySQL_DB_HOST=localhost
EXPRESS_MySQL_DB_USERNAME=root
EXPRESS_MySQL_DB_PASSWORD=
EXPRESS_MySQL_DB_NAME=pw_praktikum_db
EXPRESS_MySQL_DB_PORT=3306

// MongoDB Configuration
EXPRESS_Mongo_DB_HOST=localhost
EXPRESS_Mongo_DB_USERNAME=admin
EXPRESS_Mongo_DB_PASSWORD=password123
EXPRESS_Mongo_DB_NAME=pw_user_praktikum_db
EXPRESS_Mongo_DB_PORT=27017
```

config > config.js > ...

```js
require("dotenv").config();

// MongoDB Configuration menggunakan variabel lingkungan dengan fallback ke nilai default
const host = process.env.EXPRESS_Mongo_DB_HOST || "localhost";
const port = process.env.EXPRESS_Mongo_DB_PORT || "27017";
const name = process.env.EXPRESS_Mongo_DB_NAME || "pw_praktikum_db";
const user = process.env.EXPRESS_Mongo_DB_USERNAME;
const pass = process.env.EXPRESS_Mongo_DB_PASSWORD;

// Ekspor konfigurasi untuk berbagai lingkungan (development, test, production)
module.exports = {
    development: {
        url: `mongodb://${host}:${port}/${name}`
    },
    test: {
        url: `mongodb://${host}:${port}/${name}_test`
    },
    production: {
        url: `mongodb://${user}:${pass}@${host}:${port}/${name}?authSource=admin`
    }
};
```

models > index.js > ...

```js
'use strict';

module.exports = {
  User : require('./users'), // Ekspor model User untuk konsistensi dengan Sequelize
  Users: require('./users') // Ekspor model Users untuk konsistensi dengan Sequelize
};
```

models > users.js > ...

```
'use strict';
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
    email: {
        type: String,
        required: [true, 'Email wajib diisi'], // Pesan error jika email tidak diisi
        unique: true,
        maxlength: 100
    },
    username: {
        type: String,
        required: [true, 'Username wajib diisi'], // Pesan error jika username tidak diisi
        unique: false,
        maxlength: 50
    },
    password: {
        type: String,
        required: [true, 'Password wajib diisi'], // Pesan error jika password tidak diisi
        maxlength: 255
    },
    is_active: {
        type: Boolean, // Menggantikan tinyint(1)
        default: true  // Nilai default
    }
}, {
    timestamps: true // Otomatis membuat field createdAt dan updatedAt
});

module.exports = mongoose.model('User', userSchema);
```

migrations > 20260410152528-create-users.js > <unknown>

```javascript
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(db, client) {

    // Membuat collection 'users' dengan validasi schema
    await db.createCollection('users', {
      validator: {
        $jsonSchema: {
          bsonType: 'object',
          required: ['email', 'username', 'password'],
          properties: {
            email: {
              bsonType: 'string',
              description: 'Email wajib diisi dan harus berupa string'
            },
            username: {
              bsonType: 'string',
              description: 'Username wajib diisi dan harus berupa string'
            },
            password: {
              bsonType: 'string',
              description: 'Password wajib diisi dan harus berupa string'
            },
            is_active: {
              bsonType: 'bool',
              description: 'is_active harus berupa boolean'
            }
          }
        }
      }
    });

    // Membuat index untuk field email, username, password, dan is_active
    await db.collection('users').createIndex({ email: 1 }, { unique: true });
    await db.collection('users').createIndex({ username: 1 });
    await db.collection('users').createIndex({ password: 1 });
    await db.collection('users').createIndex({ is_active: 1 });
  },
  async down(db, client) {
    await db.collection('users').drop();
  }
};
```

controllers > userController.js > deleteUser > deleteUser

```javascript
const User = require("../models/users");

// --- User Controller ---
// Memanggil semua user data
exports.getAllUsers = async (req, res) => {
    try {
        // Menangkap query parameters untuk pencarian, pengurutan, dan urutan
        const { search, sort_by, order } = req.query;

        // Membangun query pencarian jika parameter search ada
        let query = {};
        if (search) {
            query = {
                $or: [
                    { username: { $regex: search, $options: 'i' } },
                    { email: { $regex: search, $options: 'i' } }
                ]
            };
        }

        //Membangun objek sort berdasarkan parameter sort_by dan order
        let sortObj = { createdAt: -1 };
        if (sort_by) {
            const sortOrder = order && order.toUpperCase() === 'ASC' ? 1 : -1;
            sortObj = { [sort_by]: sortOrder };
        }

        const users = await User.find(query).sort(sortObj);

        res.json({
            success: true,
            message: "Users fetched successfully",
            data: users
        });
    } catch (error) {
        console.error("Error fetching users:", error);
        res.status(500).json({
            success: false,
            message: "Failed to fetch users",
            error: error.message
        });
    }
};

```

controllers > userController.js > deleteUser > deleteUser

D:\Kuliah\PWL\UserManagements\routes\userRoutes.js

```javascript
};

// Menangkap user data berdasarkan id
exports.getUserById = async (req, res) => {
    try {
        // Mencari user berdasarkan ID yang diberikan dalam parameter URL
        const user = await User.findById(req.params.id);

        if (!user) {
            return res.status(404).json({
                success: false,
                message: "User not found"
            });
        }

        res.json({
            success: true,
            message: "User fetched successfully",
            data: user
        });
    } catch (error) {
        console.error("Error fetching user:", error);
        res.status(500).json({
            success: false,
            message: "Failed to fetch user",
            error: error.message
        });
    }
};

```

controllers > userController.js > deleteUser > deleteUser

```javascript

// Membuat user baru
exports.createUser = async (req, res) => {
    try {
        // Menangkap data yang dikirim melalui body request
        const { username, email, password, is_active } = req.body;

        if (!username || !email || !password) {
            return res.status(400).json({
                success: false,
                message: "Username, email, and password are required"
            });
        }

        const user = await User.create({
            username,
            email,
            password,
            is_active: is_active !== undefined ? is_active : true
        });

        res.status(201).json({
            success: true,
            message: "User created successfully",
            data: user
        });
    } catch (error) {
        console.error("Error creating user:", error);

        // Menangani error validasi dari Mongoose
        if (error.name === 'ValidationError') {
            return res.status(400).json({
                success: false,
                message: "Validation error",
                errors: Object.values(error.errors).map(e => ({
                    field: e.path,
                    message: e.message
                }))
            });
        }
        res.status(500).json({
            success: false,
            message: "Failed to create user",
            error: error.message
        });
    }
};

```

controllers > userController.js > deleteUser > deleteUser

```javascript


// Memperbarui data user berdasarkan id
exports.updateUser = async (req, res) => {
    try {
        // Mencari user berdasarkan ID yang diberikan dalam parameter URL
        const user = await User.findById(req.params.id);

        if (!user) {
            return res.status(404).json({
                success: false,
                message: "User not found"
            });
        }

        // Menangkap data yang dikirim melalui body request
        const { username, email, password, is_active } = req.body;

        if (username !== undefined) user.username = username;
        if (email !== undefined) user.email = email;
        if (password !== undefined) user.password = password;
        if (is_active !== undefined) user.is_active = is_active;

        await user.save();

        res.json({
            success: true,
            message: "User updated successfully",
            data: user
        });
    } catch (error) {
        console.error("Error updating user:", error);

        if (error.name === 'ValidationError') {
            return res.status(400).json({
                success: false,
                message: "Validation error",
                errors: Object.values(error.errors).map(e => ({
                    field: e.path,
                    message: e.message
                }))
            });
        }
        res.status(500).json({
            success: false,
            message: "Failed to update user",
            error: error.message
        });
    }
};

```

controllers > userController.js > deleteUser > deleteUser

```javascript
// Menghapus user berdasarkan id
exports.deleteUser = async (req, res) => {
    try {
        // Mencari user berdasarkan ID yang diberikan dalam parameter URL dan menghapusnya
        const user = await User.findByIdAndDelete(req.params.id);

        if (!user) {
            return res.status(404).json({
                success: false,
                message: "User not found"
            });
        }

        res.json({
            success: true,
            message: "User deleted successfully"
        });
    } catch (error) {
        console.error("Error deleting user:", error);
        res.status(500).json({
            success: false,
            message: "Failed to delete user",
            error: error.message
        });
    }
};
```

index.js > ...

```javascript
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const mongoose = require('mongoose');
const config = require('./config/config.js'); // Memanggil file config yang baru

const app = express();
const env = process.env.NODE_ENV || 'development';

// Middleware
app.use(cors({
    origin: "http://localhost:3000",
    credentials: true
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

const mongoURI = config[env].url;

mongoose.connect(mongoURI)
  .then(() => {
      console.log(`✓ Koneksi ke MongoDB berhasil (Mode: ${env})`);
  })
  .catch((err) => {
      console.error('❌ Gagal terhubung ke MongoDB:', err.message);
  });


// --- Basic Routes ---
app.get("/", (req, res) => {
    res.json({
        message: "Server berjalan dengan baik",
        status: "active",
        timestamp: new Date()
    });
});

app.get("/api/info", (req, res) => {
    res.json({
        message: "API User Management Built with Express.js & MongoDB", // Teks MySQL dihapus
        version: "1.0.0",
        status: "active",
        database: "Connected with Mongoose", // Teks Sequelize dihapus
        environment: env, // Menambahkan info environment untuk memudahkan debugging
        timestamp: new Date()
    });
});
```

index.js > ...

```javascript
// --- API Routes ---
const userRoutes = require("./routes/userRoutes");
app.use("/api/users", userRoutes);

// --- 404 Handler ---
app.use((req, res) => {
    res.status(404).json({
        success: false,
        message: "Route not found"
    });
});

// --- Error Handler ---
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({
        success: false,
        message: "Internal server error",
        error: err.message
    });
});

// --- Start Server ---
// Menggunakan API_PORT sesuai dengan format .env yang kita bahas
const PORT = process.env.API_PORT || process.env.PORT || 3002;

app.listen(PORT, () => {
    console.log(`✓ Server running on port ${PORT}`);
    console.log(`✓ API available at http://localhost:${PORT}`);
});
```

routes > userRoutes.js > ...

```javascript
const express = require("express");
const router = express.Router();
const userController = require("../controllers/userController");

// --- User Routes ---
router.get("/", userController.getAllUsers);
router.get("/:id", userController.getUserById);
router.post("/", userController.createUser);
router.put("/:id", userController.updateUser);
router.delete("/:id", userController.deleteUser);

module.exports = router;
```

Hasil Pengujian API pada Postman

- Get All Users

GET http://localhost:3002/api/users/ Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 10 Test Results 200 OK 8 ms 1.21 KB Save Response

{} JSON Preview Visualize


```
{
    "success": true,
    "message": "Users fetched successfully",
    "data": [
        {
            "_id": "69da9a6547846aa4b5393aa1",
            "email": "victor@example.com",
            "username": "Victpr Thunder",
            "password": "5555",
            "is_active": true,
            "createdAt": "2026-04-11T19:00:53.480Z",
            "updatedAt": "2026-04-11T19:00:53.480Z",
            "__v": 0
        },
        {
            "_id": "69da9a6047846aa4b5393aa0",
            "email": "jane@example.com",
            "username": "Mary Jane",
            "password": "21121",
            "is_active": true,
            "createdAt": "2026-04-11T19:00:48.900Z",
            "updatedAt": "2026-04-11T19:00:48.900Z",
            "__v": 0
        },
```


- Get User by Id

GET http://localhost:3002/api/users/ :id Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Path Variables

Key	Value	Description	Bulk Edit
id	69da9a6547846aa4b5393aa1	Description	

Body Cookies Headers 10 Test Results 200 OK 57 ms 617 B Save Response

{} JSON Preview Visualize


```
{
    "success": true,
    "message": "User fetched successfully",
    "data": {
        "_id": "69da9a6547846aa4b5393aa1",
        "email": "victor@example.com",
        "username": "Victpr Thunder",
        "password": "5555",
        "is_active": true,
        "createdAt": "2026-04-11T19:00:53.480Z",
        "updatedAt": "2026-04-11T19:00:53.480Z",
        "__v": 0
    }
}
```

- Create User

POST http://localhost:3002/api/users/ Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
    "email": "yeager@example.com",
    "username": "Eren Yeager",
    "password": "2345678912345",
    "is_active": false
}
```

Body Cookies Headers 10 Test Results 201 Created 47 ms 629 B Save Response

JSON Preview Visualize

```
{
    "success": true,
    "message": "User created successfully",
    "data": {
        "email": "yeager@example.com",
        "username": "Eren Yeager",
        "password": "2345678912345",
        "is_active": false,
        "_id": "69dbcff357a9455b68079172",
        "createdAt": "2026-04-12T17:01:39.757Z",
        "updatedAt": "2026-04-12T17:01:39.757Z",
        "__v": 0
    }
}
```

- Update User

PUT http://localhost:3002/api/users/:id Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

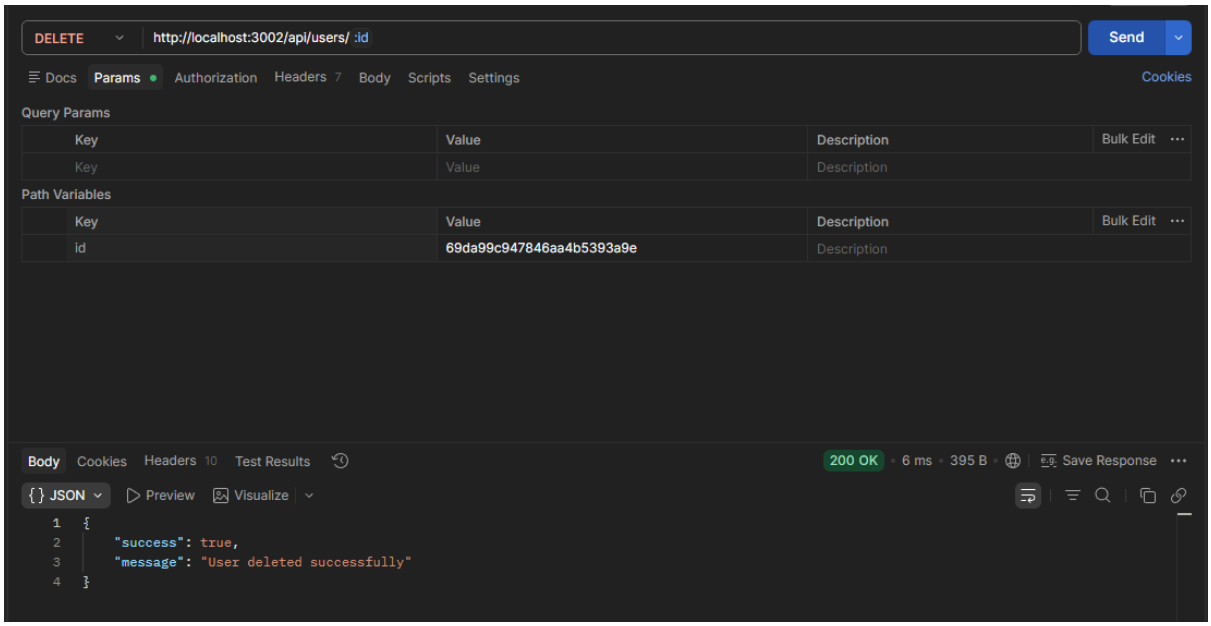
```
{
    "email": "shoto@gmail.org",
    "username": "Todoroki Shoto",
    "password": "T_1234567"
}
```

Body Cookies Headers 10 Test Results 200 OK 11 ms 619 B Save Response

JSON Preview Visualize

```
{
    "success": true,
    "message": "User updated successfully",
    "data": {
        "_id": "69da9a6047846aa4b5393aa0",
        "email": "shoto@gmail.org",
        "username": "Todoroki Shoto",
        "password": "T_1234567",
        "is_active": true,
        "createdAt": "2026-04-11T19:00:48.900Z",
        "updatedAt": "2026-04-12T17:03:49.998Z",
        "__v": 0
    }
}
```

- Delete User


DELETE
http://localhost:3002/api/users/ :id
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Path Variables
id
69da99c947846aa4b5393a9e
Body
Cookies
Headers 10
Test Results
200 OK
6 ms
395 B
Save Response
JSON
Preview
Visualize
{
"success": true,
"message": "User deleted successfully"
}